DATA SCIENCE - 22DS102006

10. Case Study: Text Emotion Detection

Detect emotional tones in short text messages using machine learning.

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB

# Sample data
texts = ['I am happy', 'I am angry', 'I feel sad', 'I love this']
labels = ['happy', 'angry', 'sad', 'happy']

# Vectorization
vec = CountVectorizer()
X = vec.fit_transform(texts)

# Train model
model = MultinomialNB()
model.fit(X, labels)

# Test prediction
test = vec.transform(['I am so sad today'])
print(model.predict(test))
```

You can enhance this by using pre-trained transformers like BERT or emoji-mapping dictionaries.

Case Study 1: Text Emotions Detection

- **Use Case: A human can express his emotions in any form, such as the face, gestures, speech and text. The detection of text emotions is a content-based classification problem. Detecting a person's emotions is a difficult task, but detecting the emotions using text written by a person is even more difficult as a human can express his emotions in any form.**
- **Recognizing this type of emotion from a text written by a person plays an important role in applications such as chatbots, customer support forum, customer reviews etc. So you have to train a machine learning model that can identify the emotion of a text by presenting the most relevant emoji according to the input text.**

Case Study 1: Text Emotions Detection

1. Aim

To develop a Machine Learning model that identifies the emotion expressed in a text message and displays a suitable emoji based on the detected emotion.

2. Problem Statement

People express emotions through text messages, social media posts, customer reviews, and online conversations. However, understanding emotions from text is challenging because the same words can have different meanings depending on the context.

The objective of this project is to build a text emotion detection system that classifies input text into categories such as happiness, sadness, anger, love, fear, and surprise, and displays the most relevant emoji for the predicted emotion.

3. Proposed Solution

The system uses Natural Language Processing (NLP) and Machine Learning to analyze text and predict its emotional category.

Input text	Detected emotion	Emoji
I am so happy today!	Happy	😊
I miss my best friend.	Sadness	😢
I am very angry with you.	Anger	😠
I love spending time with you.	Love	❤️
I am scared of the dark.	Fear	😨
Wow, this is amazing!	Surprise	😲

These are illustrative examples of expected predictions.

4. Dataset Collection

A labeled emotion dataset is required to train the model. Each text example should have an emotion label.

Example:

Text	Emotion label
I am feeling great today	Happy
I feel lonely and upset	Sadness
This makes me furious	Anger
I am afraid of the result	Fear
You mean everything to me	Love
I cannot believe this happened	Surprise

The dataset is divided into training data and testing data. The training data teaches the model to recognize emotion patterns, while the testing data evaluates its performance on unseen text.

5. Text Preprocessing

Before training, the text is cleaned and converted into a format suitable for machine learning.

1. Convert text to lowercase.
2. Remove unnecessary spaces and unwanted characters.
3. Convert text into numerical features using TF-IDF (Term Frequency–Inverse Document Frequency).
4. Preserve meaningful words and negations such as “not happy,” because they can change the emotion.
5. Split the dataset into training and testing sets.

6. Model Training and Python Code

This example uses a TF-IDF vectorizer and a Logistic Regression classifier. It includes sample training data so that the program can run directly without downloading a dataset. For a real project, replace the sample data with a larger, labeled emotion dataset.

Text Emotion Detection with Emoji Prediction

```
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score,
classification_report

# 1. Sample labeled dataset
data = {
    "text": [
        "I am very happy today",
        "This is a wonderful day",
        "I feel cheerful and excited",
        "I am sad and lonely",
        "I feel very disappointed",
        "I miss my family so much",
        "I am angry with you",
        "This makes me furious",
        "I hate this situation",
        "I am scared of the dark",
        "I am afraid of losing",
        "I feel nervous and frightened",
        "I love you so much",
        "You mean everything to me",
        "I adore spending time with you",
        "Wow this is unbelievable",
        "I did not expect this surprise",
        "What an amazing unexpected gift"
    ],
    "emotion": [
        "happy", "happy", "happy",
        "sadness", "sadness", "sadness",
        "anger", "anger", "anger",
        "fear", "fear", "fear",
        "love", "love", "love",
        "surprise", "surprise", "surprise"
    ]
}

df = pd.DataFrame(data)

# 2. Split the dataset
X_train, X_test, y_train, y_test = train_test_split(
```

```
    df["text"],
    df["emotion"],
    test_size=0.33,
    random_state=42,
    stratify=df["emotion"]
)

# 3. Build the machine learning pipeline
model = Pipeline([
    ("tfidf", TfidfVectorizer(ngram_range=(1, 2))),
    ("classifier", LogisticRegression(max_iter=1000))
])

# 4. Train the model
model.fit(X_train, y_train)

# 5. Evaluate the model
predictions = model.predict(X_test)

print("Test Accuracy:",
      round(accuracy_score(y_test, predictions) * 100, 2),
"%")

print("\nClassification Report:")
print(classification_report(
    y_test, predictions, zero_division=0
))

# 6. Map emotions to emojis
emoji_map = {
    "happy": "😊",
    "sadness": "😢",
    "anger": "😠",
    "fear": "😨",
    "love": "❤️",
    "surprise": "😲"
}

# 7. Predict emotion for new text
def detect_emotion(text):
    emotion = model.predict([text])[0]
    emoji = emoji_map[emotion]

    print("Input Text:", text)
    print("Detected Emotion:", emotion)
    print("Relevant Emoji:", emoji)
```

```
# 8. Test the application
detect_emotion("I am happy to see you")
detect_emotion("I feel lonely today")
detect_emotion("I love my family")
```

Output:

7. Results and Explanation

The system produces three main outputs for every input: the original text, the predicted emotion, and the corresponding emoji.

Example of the expected output format:

😊

Input: I am happy to see you

Predicted emotion: Happy

😢

Input: I feel lonely today

Predicted emotion: Sadness

❤️

Input: I love my family

Predicted emotion: Love

These examples show the intended behavior, not verified outputs from the trained model.

Explanation of the results

- Happy 😊: The model identifies positive expressions such as “happy,” “wonderful,” and “cheerful.”
- Sadness 😢: It recognizes expressions associated with loneliness, disappointment, or missing someone.
- Anger 😠: It identifies words associated with frustration and anger.
- Fear 😨: It recognizes expressions related to nervousness, fear, and uncertainty.
- Love ❤️: It identifies affectionate and caring expressions.
- Surprise 😲: It recognizes expressions indicating unexpected events or amazement.

The model learns patterns from labeled examples and predicts the most likely emotion among the categories it was trained on.

8. Performance Evaluation

The model should be evaluated using the following metrics:

Metric	Purpose
Accuracy	Measures the proportion of correctly classified texts
Precision	Measures how often predictions for an emotion are correct
Recall	Measures how many actual examples of an emotion are identified
F1-score	Balances precision and recall
Confusion matrix	Shows which emotions are commonly confused

A larger, balanced dataset is important because emotions such as sadness, anger, and fear can have similar expressions. Testing on unseen examples helps determine whether the model has learned general patterns rather than memorized the training sentences.

9. Applications

1. Chatbots: Respond with more appropriate emotional awareness.
2. Customer support: Identify frustrated customers and prioritize their complaints.
3. Product reviews: Analyze positive and negative emotional feedback.
4. Social media: Summarize emotional trends in public posts.
5. Online learning: Understand students' expressed reactions to learning materials.